

Interim Project I

Anirudh

October 2025

Contents

1	Task 1: Option Pricing and P&L Simulation	2
1.1	Objective	2
1.2	Methodology	2
1.3	Summary Statistics	2
1.4	Stock Price and Option Price Evolution	3
1.5	Task 1 P&L Results	3
2	Task 2: Hedging P&L with Market Data	4
2.1	Objective	4
2.2	Data Sources	4
2.3	Parameters	4
2.4	Methodology	5
2.5	Results and Insights	5
3	Code Overview	6
3.1	Option Class	6
3.2	Option_Price Class	6
3.3	PNL_Calculator Class	6
3.4	Stock_Price_Simulator Class	7
3.5	Path_Saver Utility	7
3.6	Task 1 Main Program (<code>task1.cpp</code>)	7
3.7	Task 2 Main Program (<code>task2.cpp</code>)	8
3.8	Summary	8

1 Task 1: Option Pricing and P&L Simulation

1.1 Objective

Simulate stock price paths, compute Profit and Loss (P&L) of an option with and without hedging, and store results in CSV files for analysis.

1.2 Methodology

1. **Option Initialization:** An `Option` object is created with parameters:
 - Strike price: $K = 105$
 - Steps: $N = 100$ time steps
 - Risk-free rate: $r = 0.025$
 - Maturity: $T = 0.4$
 - Volatility: $\sigma = 0.24$
2. **Stock Path Simulation:** Using `Stock_Price_Simulator`, 1000 stock price paths are generated with 100 steps each and saved as `stock_paths.csv`.
3. **Option Pricing and P&L Computation:** For each simulated path:
 - Option prices computed using `Option.Price` for a call option.
 - `PnL.Calculator` computes:
 - Hedging error P&L (delta-hedged).
 - P&L without hedging (buy-and-hold).
4. **Data Storage:**
 - Option price trajectories: `option_prices.csv`.
 - P&L results per path: `pnl_results_task1.csv`.

1.3 Summary Statistics

Statistic	Stock Price	Option Price
Overall Mean	101.40	4.89
Overall Std Dev	10.50	5.50
Overall Min	62.25	0.00
Overall Max	170.43	65.51
Mean Final	102.60	5.30
Std Dev Final	16.01	9.23

Table 1: Summary statistics of simulated stock and option prices.

1.4 Stock Price and Option Price Evolution

The simulated stock and option prices exhibit moderate volatility and a wide range of possible values. The stock price mean is 101.40 with a standard deviation of 10.50, while option prices have a mean of 4.89 and a standard deviation of 5.50. Final stock and option prices are slightly higher than the initial average, indicating a slight upward drift.

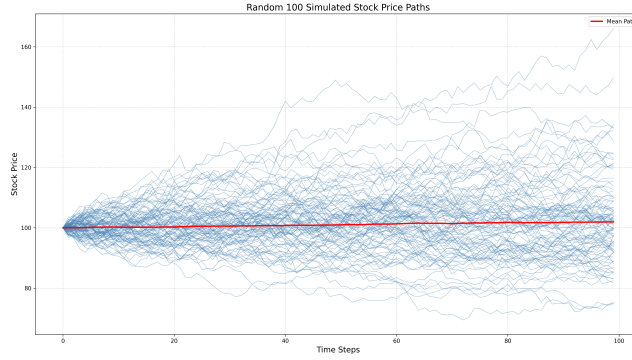


Figure 1: Simulated stock price paths from Task 1.

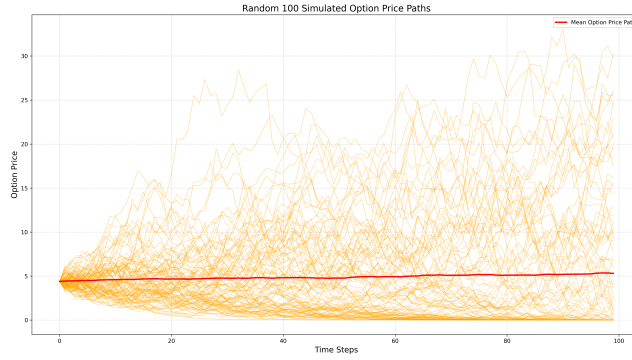


Figure 2: Option price evolution over time from Task 1.

1.5 Task 1 P&L Results

- Unhedged P&L exhibits high volatility and extreme negative outcomes, highlighting buy-and-hold risk.
- Hedging error P&L is much more stable with a mean near zero, demonstrating delta hedging effectiveness.

Statistic	PnL without Hedge	Hedging Error PnL
Mean	-0.91	0.01
Standard Deviation	9.23	0.52
Minimum	-60.77	-2.37
Maximum	4.39	2.36

Table 2: Summary statistics of P&L for Task 1, comparing unhedged vs. delta-hedged strategies.

- Maximum losses and gains are greatly limited under hedging, reducing exposure to adverse stock movements.
- Delta hedging significantly mitigates risk while keeping expected P&L near zero.

2 Task 2: Hedging P&L with Market Data

2.1 Objective

Evaluate hedging performance of a GOOG option using historical spot prices, interest rates, and option quotes to compute daily implied volatility, hedged P&L, and unhedged P&L.

2.2 Data Sources

- `sec_GOOG.csv` – daily closing spot prices.
- `interest.csv` – daily interest rates.
- `op_GOOG.csv` – option quotes with date, expiry, type, strike, bid, and ask.

2.3 Parameters

- Observation window: 07/05/2011 – 07/29/2011
- Option expiry: 09/17/2011
- Strike price: $K = 500$
- Option type: Call
- Initial volatility guess: $\sigma_0 = 0.2$
- Maturity: fraction of business days to expiry / 252

2.4 Methodology

1. Load and align spot, rate, and option data by date.
2. Compute mid-market prices and daily implied volatility.
3. Use `PNL_Calculator` to simulate hedged and unhedged P&L.
4. Store results in `task2_results.csv`.

2.5 Results and Insights

Date	Spot	MarketPrice	DailyImpliedVol	PnL_No_Hedge	PnL_Hedge
7/5/2011	532.44	44.2	0.259686	0	0
7/6/2011	535.36	46.9	0.266752	-2.0887	0.0138848
7/7/2011	546.6	55.3	0.264638	-9.99232	0.398328
7/8/2011	531.99	43.95	0.260768	2.09626	0.80036
7/11/2011	527.28	41	0.26523	5.85868	1.09732
7/12/2011	534.01	46.4	0.272777	1.12168	1.12752
7/13/2011	538.26	49.3	0.270227	-1.29202	1.89394
7/14/2011	528.94	41.15	0.253402	7.55546	3.43847
7/15/2011	597.62	99.65	0.258694	-53.8974	-6.67342
7/18/2011	594.94	97.65	0.27308	-51.2687	-6.68092
7/19/2011	602.55	103.8	0.238434	-58.4863	-6.46655
7/20/2011	595.35	97.8	0.265737	-51.3719	-6.52837
7/21/2011	606.99	108.15	0.240351	-62.8678	-6.5087
7/22/2011	618.23	118.7	0.210499	-74.0885	-6.51279
7/25/2011	618.98	119.95	0.249444	-74.827	-6.51311
7/26/2011	622.52	123.25	0.238688	-78.3551	-6.51313
7/27/2011	607.22	108.65	0.252123	-63.0445	-6.51503
7/28/2011	610.94	112.1	0.246242	-66.7523	-6.51519

Figure 3: Task 2 hedging P&L results for GOOG.

- Hedging reduces extreme P&L fluctuations; for example, large losses in `PnL_No_Hedge` on 7/15/2011 and 7/22/2011 are mitigated under `PnL_Hedge`.
- Hedged P&L remains stable across the period, while unhedged P&L is highly volatile.
- Daily implied volatility ranges from 0.21 to 0.27, reflecting market fluctuations.
- Delta hedging significantly reduces exposure but cannot eliminate all losses.
- Practical insight: dynamic delta hedging improves stability, but discrete rebalancing and transaction costs must be considered.

3 Code Overview

This section provides a detailed description of the C++ classes and code structure used for Task 1 and Task 2, including their purpose, key methods, and interactions.

3.1 Option Class

The `Option` class defines the properties of a European-style option:

- **Attributes:** Strike price (K), underlying spot price (S), risk-free rate (r), time to maturity (T), and volatility (σ).
- **Constructors:**
 - Default constructor initializes to default values.
 - Parameterized constructor allows setting custom option parameters.
- **Methods:** Getters for each attribute (`getStrike()`, `getSpot()`, `getRate()`, `getMaturity()`, `getVolatility()`).
- **Purpose:** Provides a clean interface for storing and accessing option parameters used in pricing and hedging calculations.

3.2 Option_Price Class

The `Option_Price` class implements Black-Scholes-Merton pricing and related calculations:

- **Constructor:** Takes an option type flag ("C" for Call, "P" for Put).
- **Methods:**
 - `BSM.Pricer(Option option)`: Computes option price and delta using the BSM formula. Handles edge cases such as zero time to maturity.
 - `implied_volatility(Option option, double market_price)`: Computes implied volatility using a binary search approach. Ensures market prices are within achievable bounds.
- **Purpose:** Provides both theoretical pricing and calibration to market data for use in hedging simulations.

3.3 PNL_Calculator Class

The `PNL_Calculator` class computes hedging performance and P&L:

- **Constructor:** Takes an `Option`, an `Option_Price` pricer, and the number of hedging steps.

- **Methods:**
 - `calculate_hedging_error`: Computes the P&L from a delta-hedged strategy along a simulated or historical stock path.
 - `calculate_pnl_no_hedge`: Computes the P&L for a buy-and-hold (unhedged) strategy using stored option prices.
 - `get_option_prices`: Returns the sequence of option prices along the path.
- **Purpose:** Quantifies the effectiveness of hedging strategies by comparing hedged and unhedged P&L along simulated or real market paths.

3.4 Stock_Price_Simulator Class

The `Stock_Price_Simulator` class generates stock price paths:

- **Constructor:** Accepts an `Option`, number of paths, and number of time steps.
- **Method:**
 - `simulate()`: Uses a discretized stochastic process to generate paths with geometric Brownian motion-like dynamics.
- **Purpose:** Provides input stock price trajectories for option pricing and hedging simulations in Task 1.

3.5 Path_Saver Utility

`Path_Saver` is a utility for writing simulated paths and option prices to CSV files:

- **Method:** `save_to_csv(paths, filename)` writes a 2D vector of doubles to a CSV file.
- **Purpose:** Facilitates storing and later analysis of simulation results.

3.6 Task 1 Main Program (task1.cpp)

- Initializes an `Option` and simulates stock price paths using `Stock_Price_Simulator`.
- Uses `Option_Price` and `PNL_Calculator` to compute hedged and unhedged P&L for each path.
- Saves stock paths, option price trajectories, and P&L results to CSV files.
- **Purpose:** Provides a full framework for testing delta hedging performance under simulated conditions.

3.7 Task 2 Main Program (`task2.cpp`)

- Loads historical spot prices, interest rates, and option market quotes for GOOG.
- Computes daily implied volatility for each trading day using `Option_Price::implied_volatility`.
- Uses `PNL_Calculator` to simulate hedging and no-hedge P&L along real market paths.
- Writes results including date, spot, market price, daily implied volatility, and P&L to `task2_results.csv`.
- Purpose: Tests delta hedging strategies in real market conditions, highlighting practical P&L outcomes.

3.8 Summary

The overall code framework is modular and object-oriented, with clear separation of concerns:

- `Option` defines the product.
- `Option_Price` handles pricing and implied volatility.
- `PNL_Calculator` computes performance metrics.
- `Stock_Price_Simulator` and CSV utilities handle data generation and storage.
- Task 1 focuses on simulated data, Task 2 on real market data.